

LAB : *Agent avec LangGraph*

Table des matières

PARTIE 1 : LLM local avec outils (Tools)	2
PARTIE 2 : Agent comme nœud de LangGraph	3
PARTIE 3 : Un workflow LangGraph qui génère un texte et s'arrête pour validation humaine (HITL) ...	5
PARTIE 4 : TP.....	6

PARTIE 1 : LLM local avec outils (Tools)

Créer un fichier tools_setup.py

```
from langchain.tools import tool
from langchain_ollama import ChatOllama
from dotenv import load_dotenv

load_dotenv() # si openai , groq ...

model = ChatOllama(
    model="llama3.2:3b", # ou mistral, gemma, etc.
)

@tool
def add(a: int, b: int) -> int:
    """Add two integers."""
    return a + b

@tool
def multiply(a: int, b: int) -> int:
    """multiply two integers."""
    return a * b

@tool
def divide(a: int, b: int) -> float:
    """divide two integers."""
    return a / b

tools = [add, multiply, divide]
tools_by_name = {t.name: t for t in tools}

model_with_tools = model.bind_tools(tools)
```

PARTIE 2 : Agent comme nœud de LangGraph

Construire un agent avec outils (tools) basé sur LangGraph qui :

- reçoit une question utilisateur
- laisse le LLM décider s'il doit utiliser un tool
- exécute le tool si nécessaire
- renvoie la réponse finale
- peut fonctionner en streaming

L'état contient :

messages

- historique complet de la conversation
- utilise add pour concaténation automatique

llm_calls

- compteur du nombre d'appels au LLM

Le résultat :

- ajoute la réponse du LLM dans messages
- incrémente compteur

Créer un fichier agent_node.py

```
from operator import add
from typing import Literal
from typing_extensions import TypedDict, Annotated
from langchain_core.messages import AnyMessage
from langchain.messages import SystemMessage, ToolMessage, HumanMessage

from langgraph.graph import StateGraph, START, END

from tools_setup import model_with_tools, tools_by_name

class AgentState(TypedDict):
    messages: Annotated[list[AnyMessage], add]
    llm_calls: int

def llm_call(state: AgentState):
    """LLM decides whether to call tools or respond."""
    response = model_with_tools.invoke(
```

```

        [SystemMessage(content="You are a helpful assistant that solves arithmetic problem using tools when needed.")]
        + state["messages"]
    )

    return {
        "messages": [response],
        "llm_calls": state.get("llm_calls", 0) + 1,
    }

def tool_node(state: AgentState):
    """Execute tool calls from last AI message."""
    last = state["messages"][-1]
    results = []

    for call in last.tool_calls:
        tool = tools_by_name[call["name"]]
        observation = tool.invoke(call["args"])
        results.append(ToolMessage(content=str(observation),
tool_call_id=call["id"]))

    return {"messages": results}

def should_continue(state: AgentState) -> Literal["tool_node", END]:
    last = state["messages"][-1]

    if getattr(last, "tool_calls", None):
        if last.tool_calls:
            return "tool_node"
    return END

builder = StateGraph(AgentState)

builder.add_node("llm_call", llm_call)
builder.add_node("tool_node", tool_node)

builder.add_edge(START, "llm_call")
builder.add_conditional_edges("llm_call", should_continue, ["tool_node", END])
builder.add_edge("tool_node", "llm_call")

agent = builder.compile()

if __name__ == "__main__":
    result = agent.invoke({"messages": [HumanMessage(content="Add 3 and 4.")],
"llm_calls": 0})

```

```
print(result)

for m in result["messages"]: # parcourir tout l'historique conversationnel
    try:
        m.pretty_print() # affiche type du message, contenu et tool calls

    except Exception as ex:
        print(ex)
        print(m)

# Stream updates (state deltas)

for chunk in agent.stream( # chaque chunk est une modification du state
    {"messages": [HumanMessage(content="Multiple 30 and 43.")],
    "llm_calls": 0},
    stream_mode="updates", #afficher chaque modification du graphe en
    direct node par node
):
    print(chunk)

# Stream messages LLM tokens et metadata
for message_chunk, metadata in agent.stream(
    {"messages": [HumanMessage(content="Divide 30 and 43.")],
    "llm_calls": 0},
    stream_mode="messages", #les sorties du LLM et métadonnées
):
    if getattr(message_chunk, "content", None):
        print(message_chunk.content, end="", flush=True)
print()
```

PARTIE 3 : Un workflow LangGraph qui génère un texte et s'arrête pour validation humaine (HITL)

@task transforme une fonction Python en une brique d'exécution isolée dans un workflow LangGraph.

@entrypoint définit la fonction principale qui démarre le graphe. Un entrypoint :

- reçoit l'entrée utilisateur
- orchestre les tasks
- définit le flux global
- peut inclure interrupt()
- est exécuté via .stream() ou .invoke()

```
import time
import uuid
from langgraph.checkpoint.memory import InMemorySaver
from langgraph.func import entrypoint, task
from langgraph.types import interrupt, Command

@task
def write_essay(topic: str) -> str:
    time.sleep(1)
    return f"Essay draft about {topic}"

@entrypoint(checkpointer=InMemorySaver())
def workflow(topic: str) -> dict:
    draft = write_essay(topic).result()
    approved = interrupt({"draft": draft, "action": "approve or reject"})
    return {"draft": draft, "approved": approved}

thread_id = str(uuid.uuid4())

config = {"configurable": {"thread_id": thread_id}}

# Première exécution
for item in workflow.stream("cats", config):
    print(item)
# Deuxième exécution
for item in workflow.stream(Command(resume=True), config):
    print(item)
```

PARTIE 4 : TP

Ce code construit un agent LangGraph avancé capable de :

- utiliser des tools
- s'arrêter pour validation humaine
- reprendre après interruption
- sauvegarder l'historique complet
- revenir dans le passé et modifier un état (fork)

```
from langgraph.checkpoint.memory import InMemorySaver
from langchain.messages import HumanMessage

from operator import add
from typing import Literal
from typing_extensions import TypedDict, Annotated
from langchain_core.messages import AnyMessage
from langchain.messages import SystemMessage, ToolMessage, HumanMessage

from langgraph.graph import StateGraph, START, END
from langgraph.types import interrupt, Command
from langgraph.func import entrypoint
from langgraph.store.base import BaseStore

from tools_setup import model_with_tools, tools_by_name

@entrypoint(store=...) # pass a store instance here
def workflow(user_input: dict, *, store: BaseStore):
    # store.get(...) store.put(...) style operations happens here
    return {"ok": True}

class AgentState(TypedDict):
    messages: Annotated[list[AnyMessage], add]
    llm_calls: int

def llm_call(state: AgentState):
    """LLM decides whether to call tools or respond."""
    response = model_with_tools.invoke(
        [SystemMessage(content="You are a helpful assistant that solves arithmetic problem using tools when needed.")]
        + state["messages"]
    )

    return {
        "messages": [response],
        "llm_calls": state.get("llm_calls", 0) + 1,
    }

def tool_node(state: AgentState):
    """Execute tool calls from last AI message."""
    last = state["messages"][-1]
    results = []
```

```

    for call in last.tool_calls:
        tool = tools_by_name[call["name"]]
        observation = tool.invoke(call["args"])
        results.append(ToolMessage(content=str(observation),
tool_call_id=call["id"]))

    return {"messages": results}

def should_continue(state: AgentState) -> Literal["approve", END]:
    last = state["messages"][-1]

    if getattr(last, "tool_calls", None) and last.tool_calls:
        return "approve"
    return END

def approve_node(state: AgentState) -> Command[Literal["tool_node",
    "__end__"]]:
    decision = interrupt({"question": "Approve tool execution?", "tool_calls":
state["messages"][-1].tool_calls})
    return Command(goto="tool_node" if decision else END)

builder = StateGraph(AgentState)

builder.add_node("llm_call", llm_call)
builder.add_node("approve", approve_node)
builder.add_node("tool_node", tool_node)

checkpointer = InMemorySaver()

builder.add_edge(START, "llm_call")
builder.add_conditional_edges("llm_call", should_continue, ["approve", END])
builder.add_edge("tool_node", "llm_call")

agent = builder.compile(checkpointer=checkpointer)

config = {"configurable": {"thread_id": "thread-1"}}

result = agent.invoke({"messages": [HumanMessage(content="Add 3 and 4.")],
    "llm_calls": 0}, config=config)

# print("Final:", result["messages"][-1])
#
# snap = agent.get_state(config)
#
# print("Latest checkpoint next:", snap.next)
#

```



```
# history = list(agent.get_state_history(config))
# print("Number of checkpoints:", len(history))
# print("Most recent checkpoint id:", history[0].config["configurable"]["checkpoint_id"])

print("Interrupt payload:", result["__interrupt__"][0].value)

resume = agent.invoke(Command(resume=True), config=config)
print("Done. Last message:", resume["messages"][-1])

# print("Final:", result["messages"][-1])
#
# snap = agent.get_state(config)
#
# print("Latest checkpoint next:", snap.next)
#
# history = list(agent.get_state_history(config))
# print("Number of checkpoints:", len(history))
# print("Most recent checkpoint id:", history[0].config["configurable"]["checkpoint_id"])

config_reject = {"configurable": {"thread_id": "thread-1-reject"}}

result = agent.invoke({"messages": [HumanMessage(content="Multiply 30 and 41.")], "llm_calls": 0}, config=config_reject)

print("Interrupt payload:", result["__interrupt__"][0].value)

resume = agent.invoke(Command(resume=False), config=config_reject)
print("Done. Last message:", resume["messages"][-1])

history = list(agent.get_state_history(config_reject))
picked = history[1]

new_config = agent.update_state(picked.config,
                                values={"messages": [HumanMessage(content="Multiply 30 and 41.")], "llm_calls": 0})
forked = agent.invoke(None, new_config) #relance le graphe depuis un point historique
print("Forked:", forked)
```